

## some Summer Research Project Logbook

### Week 1

Day 1 | 24/11/25 | 7.5 hrs

- <https://www.intel.com/content/www/us/en/support/programmable/support-resources/devices/arria-10-sx-support.html>
- Useful resources found for continuing the project such as the FGPA AI Suite and the OpenVINO Toolkit as I will ideally be switching to the Intel Aria 10 board instead of continuing with the cyclone.
- FPGA AI Suite already uses a built in IP for deploying AI inference onto the FPGA but since I already have designed my own custom architecture, I wont exactly be needing that. However, it does provide a more structured end-to-end pathway that may be of use as inspiration, especially on how to use the config files that can generated by the OpenVINO.
- OpenVINO (Open Visual Inference and Neural Network Optimisation) Toolkit is a software suite that is designed to accelerate AI inference across several platforms such as the FPGA that I am using.
  - I would be able to use NIOS II or could even migrate to NIOS V using the Heterogeneous Plugin to delegate my preprocessing to that coprocessor or just make NIOS host the OpenVino Runtime which would essentially become my overall Control Unit, which would manage the memory transactions and instructions for my NPU architecture that I have designed.
  - Would most likely use C++ however, the framework will need to be re-evaluated since AlexNet was used previously on Python script.

Potential Revamps based upon upgrade to Aria 10:

- Number of DSP blocks will be upgraded from 81 to ~1600
- M10K on chip memory to M20K blocks which would an increase from ~6mb to ~48mb

To proceed with the idea of using OpenVINO I would most likely have to use my arch linux OS on my computer to make it work with a C++ based framework instead of the AlexNet that I had on python. For that I would also need to install Intel Quartus Prime Pro

I have found a paper that discusses the analysis of the acceleration of deep learning inference models on a heterogenous architecture based on OpenVINO. This could help justify the use of the OpenVINO software and potentially help in using it.

<https://ieeexplore.ieee.org/document/9668607/citations#citations>

The paper discusses the use of the intel arria 10 FPGA alongside CPU or CNN inferencing compared to just a standard CPU inference, as that is extremely helpful when operating at the edge. Nothing that I already don't know. The most important part is the OpenVINO framework. It comprises of two parts: the intermediate representation (IR) which is the generated via Model Optimiser, converting pretrained model into .bin and .xml. .bin is binary file consisting of weights and biases, and .xml shows the structure of the layers and connectivity. The second part is the inference engine that consists of the C++ or python library which would provide the API for reading the intermediate representation.

## Day 2 | 25/11/25 | 5 Hrs

- Spent most of the time trying to fix errors to download the OpenVINO software on linux. Multiple errors during installation of C++ and python libraries and other version conflicts on my linux system.

Will continue on the research aspect of things to prepare further for when I get access to the Arria 10 FPGA board.

There is a hardware acceleration paper that discusses optimisation techniques when accelerating convolution operations on an Arria 10 FPGA board:

<https://ieeexplore.ieee.org/document/8330049>

### 1. Loop Unrolling (Parallelism)

- The main purpose of this is to maximise the parallelism of computations by mapping multiple loop iterations directly into dedicated hardware. This optimisation is crucial because it sets the factor by which computational speed is increased. How?
  - It designs the PE grid to perform MAC operations simultaneously per clock cycle
  - The degree of unrolling is defined by the set of design variables denoted as  $P^*$
  - The total number of parallel MAC operations ( $P_m$ ) is the product of these variables, which directly determines maximum speed ( $\text{Throughput} = N_m / P_m$ ).
  - Unrolling specific loops is necessary to exploit data reuse, which is vital for minimising memory access.
    - Unrolling Loop-3 ( $P_{ix} \times P_{iy}$ ) is the method used to achieve spatial reuse of weights, where one weight.
    - Unrolling Loop-4 ( $P_{of}$ ) is key to realising spatial reuse of input pixels/features.

### 2. Loop Tiling (Memory Partitioning)

- Loop tiling is necessary because the on-chip memory of an FPGA is insufficient to store the large data volumes associated with deep CNNs, requiring gigabytes of external memory (SDRAM).
  - Tiling divides the model's entire data set (pixels, weights, and intermediate results) into multiple smaller blocks that are sized specifically to fit within the limited capacity of the on-chip buffers.
  - The extent of tiling is governed by the  $T^*$  variables. These variables define the exact dimensions of the chunk of data loaded from external memory into the on-chip buffers.
  - The tiling variables determine the lower bound on the required size of the on-chip buffers.
  - Proper tiling is the primary mechanism for reducing the number of costly DRAM accesses, which in turn minimises latency and power consumption.

### 3. Loop interchange (Latency Factor)

- Loop interchange determines the flow and execution sequence of the convolution loops and is critical for managing transient data storage.
  - Defines the sequential order in which the four convolution loops are computed. This decision dictates the pattern of data movement both within the on-chip buffer to the PEs and from the external memory to the on-chip memory.
  - The primary objective of loop interchange is to minimise the number and movement of partial sums. A partial sum is the intermediate result of a MAC operation that must be stored until the final accumulation is complete.

- This optimisation is not very necessary to me because my existing design is an output stationary orthogonal architecture of Pes so I don't exactly have any partial sums to deal with.

### Day 3 | 26/11/25 | 5

Currently a bit confused about the DMA and AXI pipeline observed in the intel DMA reference design documentation. The Arria 10 supports PCIe Gen3 x8 links which is up to 8Gbps rate of data transfer and uses the Avalon-MM interface of DMA within the FPGA fabric. So this means that my NPU that I have designed must communicate through AXI4 or Avalon-MM I am currently unsure. Will have to confirm once I get the board and use the platform designer.

Upon research of the RISC-V acceleration, the sparsity calculations can be accelerated by creating a single custom instruction that the NIOS V executes, replacing multiple software instructions.

<https://arxiv.org/pdf/2511.06955>

This paper presents the acceleration of deep learning on resource-constrained edge FPGAs by fusing a soft-core processor with custom hardware acceleration. The complexity of this paper lies in its quantitative analysis of bottlenecks, which is crucial for maximising the efficiency of my Arria 10 design.

The central goal of the paper to solve the trilemma of Edge AI: balancing latency, power, and flexibility. Unlike previous works that treated hardware acceleration as a slow, generic memory-mapped coprocessor, this research uses ISA extensions to create a true hardware-software co-design.

The primary innovation is integrating the accelerator at the instruction level. This means the software running on the RISC-V core does not manage the accelerator by writing multiple values to random memory addresses. Instead the CPU issues a single custom assembly instruction (e.g. FPGA.VCONV) that directly triggers the entire complex hardware operation.

The architecture itself is a self-contained embedded system built around a RISC-V core deployed on the Xilinx Zynq-7020 SoC which contains an ARM Cortex-A9 as a baseline for comparison.

| Extension | Functionality & Hardware                                                                                                                                                                          | Perofmrnce Interpretation                                                                                                                                        |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FPGA.CONV | Vectorised Convolution: The core NPU accelerator. It is a 4x4 systolic array implemented with 32 DSP slices. The instruction repalcecs the entire loop nest of the convolution kernel in software | Achieved a 7.20x speedup over the highly optimisd ARM NEON baseline. This single instruction accounts for 60% to 75% of the total time saved in the application. |
| FPGA.GEMM | General Matrix Multiplication: An 8x8 systolic array optimised for tasks like fully connected layers                                                                                              | Achieved 4.20x speedup. The 8x8 array was found to be the optimal size for maximising efficiency without saturating the memory bandwidth.                        |
| FPGA.RELU | Vectorised Activation: Handles                                                                                                                                                                    | The speedup (3x) is less                                                                                                                                         |

|             |                                                                                                                                   |                                                                                                                                                 |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
|             | non-linear activation functions (ReLU, LeakyReLU).                                                                                | dramatic but the instruction achieves an 85% reduction in instruction count. This frees up the CPU from managing repetitive scalar logic.       |
| FPGA.CUSTOM | Extensibility: An “escape hatch” providing a generic interface for specialised tasks (e.g. Non-Maximum Supression (NMS) for YOLO) | Demonstrates a critical principle: you don’t need a single monolithic NPU; specialised hardware blocks can be called via separate instructions. |

## Memory and Bottleneck Analysis (The Critical Trade-Offs)

The paper’s deep dive into hardware metrics reveals the true limitations of acceleration, which may help in my case but it is much easier since the Arria 10 has a lot more to offer in terms of memory bandwidth. The overall latency speedup was 2.14x. This measured speedup achieved only 63% of the 3.39x theoretical maximum dictated by Amdahl’s Law.

The source of inefficiency was that 15% DMA Overhead and 12% Memory Bandwidth stalls. This meant that majority of the performance ceiling is due to data transfer delays, not slow computation. This mandates solutions like DMA batching and wider data paths.

The design used 64KB of BRAM on-chip memory integrated with the AXI interconnect. Triple-buffering was found to be essential to achieve peak performance, while double-buffering caused 18% performance loss due to stalls.

The AXI4 Data Bus measured at 850 MB/s. The core challenge for this is making the accelerator ask for data efficiently. The solution is intelligent tiling and AXI burst transfers to eliminate the overhead of initiating transactions for even small data pieces.

## Day 4 | 27/11/25 | 5

### Meeting 1:

- Check OpenVino version, because latest might not work with FPGA
- Find license file, allocate the license file in bash
- something to do with WSL
- Try take advantage of the time, try to extend research skills, and possibly publish. Check out Modelzoo, so that OpenVino has access to several models not just AlexNet. It could also be used for LLMs not just CNNs.
- Try to pick out the research needs, and not explore too much unless you don’t know what you’re doing.
- Had few hours of fixing required for license errors for Quartus, I believe it is working now. The task for the rest of the day is to familiarise with the board, and possibly AI suite, OpenVINO and obviously Quartus pro.f

Plan for next week:

Upgrade the current set of Pes because currently its a 8x8 signed multiplier with a 32-bit accumulator and that is too little for maximising the use on the arria 10 board. Will aim for a 16-bit fixed point precision.

Upgrade memory system from basic ROMs to exploring the Avalon-MM/AXI4 interfaces which will facilitate later integration with the NIOS processor and DMA.

Main Questions that I need to answer:

- What exactly be the roles for each component on the software stack side of things?
- What difference between MM and AXI4?
- 

Day 5 | 01/12/25 | 4 Hours

Upgrading hardware logic according to the board. Until I faced the issue discussed in Day 6

Day 6 | 02/12/25 | 5 Hours

The large-scale 32x32 architecture immediately revealed logical shortcomings in the timing signals (ready, count) that the original 8x8 prototype did not show.

This initial problem was exposed by testing a 3x3 matrix multiplication where the latency is 7 cycles. There was a serious synchronisation and counter decoupling bug in the control unit. Basically the counter only ran for  $K=3$  cycles before the external ready signal pulsed low, prematurely stopping the count. So we removed the dependency on the external ready signal for counting and introduced a self-managing internal state (run\_enable). The counter now runs until it hits the calculated maximum latency ( $M+N+K-2$ ), independent of the external pulse duration. The input data gates were updated to rely on the internal run\_enable flag. This ensured the data streamed for the full required  $K$  cycles (32 cycles in the final test) before being safely replaced by padded zeros.

Once the core logic was fixed, the 32x32 test compiled and ran for the correct 94 cycles, however the output matrix was corrupted as it was only correct up until the 8<sup>th</sup> row and 20<sup>th</sup> column. This was discovered to be the testbench inputs being incorrect after the 8<sup>th</sup> row, so that was replaced. And now the final output is correct. Next step is to check if sparsity can be handled.

Day 7 | 04/12/2025 | 5 Hours

Row/col stripping sparsity handling does in fact work now. This has been tested with a modified testbench and recycling some old python code from p4p.

The next step is to architect a new top-level system in order to replace the previous ROM and FSM structure with a proper operational high-bandwidth memory controller and possibly some considerations with the FGPA to CPU interface suitable for my OpenVINO integration.

The current 'npu\_wrapper' and its components rely on low-bandwidth, synchronous ROM access and an FSM that loads the whole 32x32 matrix once. This is a simulation bottleneck that must be replaced by AXI/Avalon based memory hierarchy. In order to do so, a double buffer system using M20K BRAM blocks can be used to allow the accelerator to compute the current tile from Buffer A while the DMA loads the next tile into Buffer B.

Day 8 | 05/12/25 | 7 Hours

Day 9 | 08/12/25 | 7 Hours

Configured the on-chip memory to replace simulation constants in a way that maximum data throughput can be achieved. Implemented a 5 buffer system using M20K Trual Dual Port RAM blocks in a ping pong manner to maximise the locality and eliminate any NPU idle time. The two buffers for each input allow the DMA engine to write the next data tile into one buffer while the NPU reads the current tile from the other. This ensures the NPU is never stalled waiting for slow memory accesss, maintaining continuous DSP utilisation.

The buffers were designed as such:

- Two read/write ports with 16 bit width for the input buffers and 64 bit width for the output buffer. Single clock mode which just uses the CLOCK\_50 and the initial content is always stored on Port A for convenience and consistency. To ensure the system is stable upon reset and compatible with future Dynamic Partial Reconfiguration (DPR) goals, Output ACRL is selected for clearing output registers on asynchronous reset, ensuring a known zero state. Clock Enables are selected for low power to stop clock toggling. For the actual DPR there was a clock compatability selection which I just ticked.

For the M, N, K values, currently they will be hardcoded from a hardware wrapper but they will eventually be AXI registerss and will be read from those registers written a software coprocessor or even the HPS.

Is this the same as making a staging and local buffer?? – check out paper: [Bandwidth Optimisation Through On-chip memroy Restructuring for HLS](#)

Day 10 | 09/12/2025 | 7 Hours

Trying to make QuestaSim work

Day 11 | 10/12/2025 | 7 Hours

Same thing

Day 12 | 11/12/2025 | 8 Hours

Same old

Day 13 12/12/2025 | 4.5

Same issue ugghhhhh

Day 14 | 14/12/25

To discuss with Morteza:

- Need to research now more into RISC-V custom instructions, Dynamic Partial Reconfiguration, and methods for an efficient communication interface
- How to fix the questasim issue? How to make Ram2\_port library work with modelsim, if I cant make QuestaSim work?
- What happens if I don't get access to the labs during Christmas break?
- Could ask about DPS vs LUT
- Could the fpga switch dynamically between LUTs and DSPs on runtime

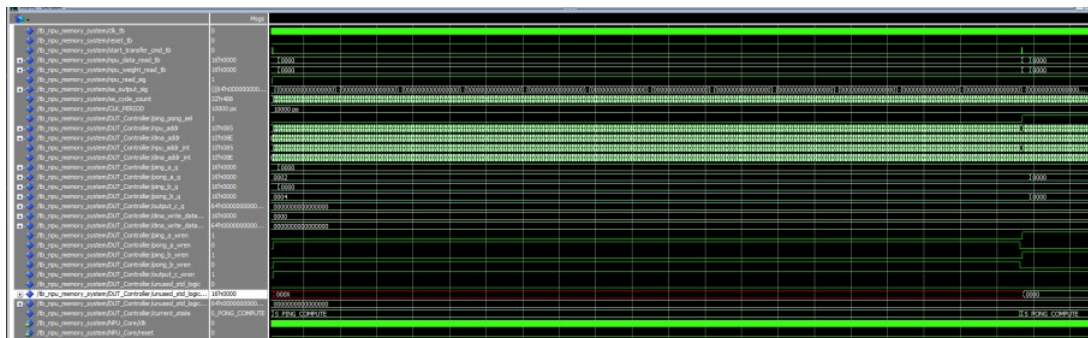
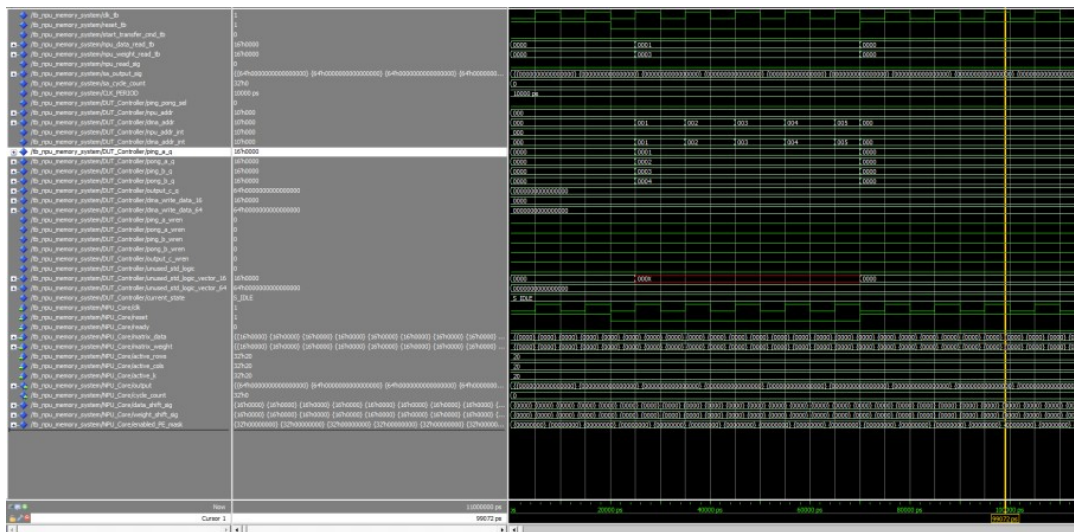
Day 15 15/12/25 | 7.5 Hours

```
vsim -t 1ps <testbench>
```

Day 16 | 16/12/25 | 8 Hours

Had a lot of errors trying to compile and run the simulation for the past week. Such as:

- Not being able to make it work on modelsim
- Altera Questa license not applicable
- Questasim not being present on lab computer
- Ram2\_port library not working but it turned out that was just a primitive ram file that was needed to be referenced as a "library" in questasim.
- After all that, finally came to a point where I could simulate the code using the testbench. But it is very buggy. As per my observations, I can see that the data buffer is being filled with the initial value and same goes for the weight buffer as soon as reset goes to 0, but as soon as it goes back high, the buffer value goes to 0 and stays that way. I can also see that that the BRAM controller FSM stays at



Day 17 17/12/25 | 2 Hours

Ideally I should be moving onto the rest of the memory hierarchy and NIOS V but before I do that, I want to explore more into the reconfigurability side of things. Previously, in the Cyclone V project, I had generate statements which allowed static reconfigurability but now is time for dynamic, which the systolic array is the perfect candidate for. To do this, Im must figure out how to partition the existing hardware design into two main areas in the FPGA floorplan where it will be the static region and reconfiograble region. The static region consists of the rest of the architecture that wouldn't need to change and the reconfigurable region would be the systolic array which would change things like the PE resolution and maybe systolic array tiling.

From my research at the beginning of the previous project, I know that there is a partial reconfiguration controller that can be made which would receive .rbf files which is the partial bitstream which is written into the region, physically changing the logic.

Its a bit time consuming because I don't have the the DMA to communicate to the manager for feeding the bitstream. So I could potentially make a simple FIFO to create a connection.



Day 18 | 18/12/25 |

Information based from what I read from the Quartus DPR User Guide:

- There are different types of partial reconfiguration, going from Hierarchical which involves multiple parent and child design partitions, static update partial reconfiguration which allows to modify a specialised region without requiring recompilation of all partitions
- Before partial reconfiguration, the fpga must be ensured to be in user mode. And then a sequence of steps occur for pr:
  1. send stop\_req to PR region
  2. Wait for stop\_ack signal to indicate PR region is ready for PR
  3. Use PR control logic to freeze all necessary outputs of the PR regions. Additionally, drive the clock enable for any initialised RAMs to a disabled state.
  4. Send the PR bitstream to the PR control block in arria 10 board to initiate the PR process
  5. On successful completion of the PR operation, reset the PR region
  6. Signal the start of PR operation by asserting the start\_req signal, and deasserting the freeze signal.
  7. Wait for the start\_ack signal to indicate that the PR region is ready for operation.
  8. Resume operation of the FPGA with the newly reconfigured PR region.

There is something called an internal host PR and external host PR. With Internal host control, and internal controller like our NIOS will communicate directly with the arria 10 PR control block. To transfer the PR bitstream into the PR control block or SDM, I would use the Avalon memory-mapped interface on the Partial Reconfiguration Controller IP core. When the device enters use mode, you initiate partial reconfiguration through the FPGA core fabric using the PR internal host.

For the external host PR, an external FPGA or CPU controls the PR reconfiguration using external dedicated pins on the target device. When using an external host, you must implement the control logic for transmission of the bitstream to the hard FPGA programming pins.

My question for now is that since it makes sense to go with internal host PR since I am using NIOS eventually, would it make sense to do that first? And also would NIOS be the one to determine the reconfigurability or would it be the future software stack part of FPGA AI Suite that would decide and how would that influence the use of partial reconfiguration?

After making the reconfigurable partition, the compiler preserves post-synthesis results for the partition and reuses the post-synthesis netlist, if no partition changes are made requiring re-synthesis. Otherwise the compiler resynthesises the partition from source files. The compiler adds wire LUTs for each interface of a Reconfigurable partition, and performs checks for PR compatibility.

Need to create something called revisions and something called partitions. - **Need to understand what they actually are in this case and their differences**

Currently in the process of making the off-chip memory and DMA controller in the platform designer, and I see that for the new versions of quartus there is a thing called modular scatter-gather DMA block. The difference with this compared to the old one is that the old one was limited with handling one transfer request at a time and has poor support for high-speed “bursting” which would be useful for

SDRAM. With the mSGDMA, NIOS V can write 10 tiny command packets into a list, and the mSGDMA will execute all 10 transfers in a row without the CPU needing to interfere again. Since I will require PR, I would need to stream the bitstream into the PR controller, for this the mSGDMA has a dedicated MM-to-Streaming that is perfect for this. But since I would also need a DMA for transferring matrices from memory into their respective buffers, I have decided to use two separate DMAs for a cleaner architecture with the mSGDMA\_Data configured as MM-to-MM and the mSGDMA\_PR configured as MM-to-Streaming for the PR controller.

Day 19 | 05/01/26 | 6 Hours

There was a lot of issues when facing the connection of stream between the DMA and PR controller due to the fact that the DMA data width was separated into 4 bytes with a total width of 32 and the PR controller was 1 singular 32 width symbol. The problem was that they could not be connected because of the difference in byte alignment and symbol alignment and the lack of changing this setting to make it compatible. However I still tried many ways to connect it such as using streaming adapters and various other middle man connections points to align the settings but the fact remained that both were different alignments.

I ended up having to change the DMA type to MM-MM instead of MM-streaming

Day 20 | 06/01/26 | 7 Hours

check if DPR is even useful because the top level systolic array file has to remain pretty much the same so like if I decide to be able to change to FP32 what happens then and also would it even change the LUT and DSPs based on the optimisation made from DPR, if not then is it even worth doing DPR, if not then is it better to try dpr for the tiling techniques.

Day 21 | 07/01/26 | 7 Hours

After I generated the platform designer system, I generated the HDL and instantiated it into the existing hardware wrapper that had the NPU core instantiation already there and was previously programmed onto the board. Since we designed the PR controller in the platform designer and planned to design the control logic in NIOS V, we needed to carve out the static and reconfigurable regions. So the main logic was set to a root partition and then set a specific design partition for reconfiguration. I also defined a logic lock region to set a physical boundary on the chip floorplan to ensure that the logic being swapped in and out would stay within a specific set of logic array blocks and not interfere with the DDR4 routing. By setting the partition to a reconfigurable type, it forces the fitter to reserve specific routing tracks and global clock buffers exclusively for that region, which reduced the total amount of “free” space available for the clock to travel from Bank 3G to memory.

With the PR regions locked down, synthesis was successful as there was minor stuff to change in the actual HDL code so it wasn't going to give much issues but fitter was throwing a fit haha. There seemed to be an I/O standard conflict because the pin was set to 1.8V, and the termination logic was differential, after doing some trial and error I made a custom assignment in the pin planner page for input termination to various options such as “OFF” or a 50 ohm with calibration or even without. With that trial and error I faced various errors that either said there was conflicts with my IO bank that

contained the LEDs and with the SDRAM component. I proceeded to then change the pin locations for the LEDs and try other stuff.

Day 22 | 08/01/26 | 7 Hours

After closer viewing of the SDRAM IP parameters, the reference clock standard is LVDS which goes to show why there was some standard clash in the fitter planning. So then I changed the clock pin to LVDS to match the standard requirement from the SDRAM. I also set an assignment for the clk as a global clock signal because the PR regions and DDR4 EMIF occupy distinct physical columns. But there seemed to be some sort of fitter deadlock and can't draw a path between them. The error said that the CLK\_50 is constrained to PIN\_F5 (IOBANK\_3G) but the IO banks used by EMIF are 3D, 3C, 3B. So if I changed the CLK\_50 to be in one of those clk pins in those banks it said that those pins are already occupied so it would give an error.

For some reason the fitter just stops at 24% and says that the requested global signal will route locally to 1 destination which seemed to be the EMIF and then it would just complain that the PLL reference clock is constrained to an IO bank outside the bank used by EMIF. This was then followed by a warning that RAM usage climbed to 2925 MB indicating some sort of memory leak caused by an infinite routing loop and then quartus would eventually crash every time because there was essentially no way to get the signal from 3G pin around the PR regions and into the hard PLL input of the DDR4 banks.

Day 23 | 09/01/26 | 5 Hours

Still spending a lot of time debugging this and no luck

Day 24 | 11/01/25 | 5 Hours

Took a step back to try and reconsider the decisions I have made or going to make for DPR. I fear that there would be a mismatch of calculations from having the PE-level DPR where my PE would switch from 16 to 8 bit resolution. Like if I have a sum that is 16-bit and then switch to 8-bit, the bit-alignment will shift, having a bunch of additional shifters would eat up LUTs unnecessarily. Also, I found out that DPR won't actually be able to toggle between LUTs and DSPs which would have been a great dynamic feature as that would increase the efficiency drastically but no you cannot transform a DSP into an LUT at runtime post synthesis.

That's why I have decided to have different partial bitstreams to account for the differences between layers or networks. When NIOS V determines the next CNN layer requires a different layer type then it may adjust accordingly.

One idea could be to have 16-bit in the earlier layers as it can be sensitive to error and require 16-bit to maintain accuracy. More deeper layers high resilient and can run on 8-bit with zero loss in accuracy. DPR can help double the throughput without re-synthesising.

Questions/Concerns for Morteza Tomorrow:

- Talk about planned design
- Talk about all the bugs occurred in platform designer and then in the fitter in quartus (which is currently still being fixed so ask Morteza how to fix). Had already fixed issues like changing the clock standard to match the SDRAM components because the 1.8V that the clock input had didn't work. But currently it gives

- Can I work on the research in March due to missing out in december and a little in january
- 

Current platform designer device model: 10AS066N3F40E2SG

Current Quartus pin planner device model: 10S066K2F40E2Sg

Day 25 | 12/01/2026 | 8 Hours

Day 26 | 13/01/2026 | 7 Hours

Day 27 | 14/01/2026 | 5 Hours

Day 28 | 15/01/2026 | 5 Hours

I have decided to take some steps back and go back to square 1 and do theoretical simulations with research as the errors and crashing facing with the build up of the DPR plans I had are just too much.

The goal is to prove that different layers in my existing AlexNet code from the previous Cyclone V project have different hardware personalities. For example, the first layer handles raw image data and is very sensitive to precision, while the middle layers can be redundant and can handle aggressive quantisation or smaller tiles.

First step is to revisit the old python code that we had in p4p and understand what's going on:

1. Quantisation: My function called `load_quantised_alexnet` takes the standard AlexNet which uses 32-bit floating point and compresses the weights into 8-bit integers, reducing the memory bandwidth by 4x.
2. Hooking: My function called `extract_conv_weights_and_activations` taps into the neural network during inference. When you pass the picture of the cat through the model, the "hook" grabs the intermediate calculations (activations) after the ReLU layer.
3. The im2col Transformation: The `F.unfold` is the most important part. Systolic arrays do Matrix Multiplication, but AlexNet does convolutions. Im2col rearranges the 3D image data into a 2D matrix so that the systolic array can process a convolution as if it were a standard dot-product.
4. Tiling: The `generate_and_save_tiles` function is there since the FPGA doesn't have enough memory to hold a 3000 x 400 matrix at once, this function chips the "Big Matrix" into little 8x8 or 16x16 tiles that fit into your BRAM buffers.

Day 29 16/01/2026 | 7 Hours

| Layer | Tile Size | Actual Cycles | Speedup |
|-------|-----------|---------------|---------|
| Conv1 | 8         | 80            | 2.3x    |
| Conv1 | 16        | 118           | 1.59x   |
| Conv2 | 8         | 121           | 1.52x   |
| Conv2 | 16        | 167           | 1.13x   |
| Conv3 | 8         | 154           | 1.19x   |
| Conv3 | 16        | 175           | 1.07x   |
| Conv4 | 8         | 170           | 1.08x   |
| Conv4 | 16        | 180           | 1.04x   |
| Conv5 | 8         | 87            | 2.11x   |
| Conv5 | 16        | 141           | 1.33x   |

This is an interesting dataset collected based on one of the images (a cat) as it does confirm the patterns that I discussed earlier. There are high speedups at the beginning and end for 8x8 tiling. This would probably because Conv 1 has the im2col expansion and the conv5 has the final feature extraction so the activations are more specialised and sparse again. Conv3/Conv4 are more dense so it would be advisable to maximise the tile to increase the throughput.

Day 30 18/01/2026 | 4 Hours

Extended dataset to whole image-set

Day 31 28/01/26 | 6 Hours

By shifting to a larger imageset and testing mutliple tiles of multiple layers of each of those images, I an determine the optimal combination of systolic array tile size and bit-precision for AlexNet layers using the sparsity stripping algorithm.

Based on the data collected, I have created 3 visualisations to help articulate these findings:

1. The best Configuration Efficiency per Layer which highlights the peak performance found for each layer.
2. The Sparsity Gain vs Tile Size which shows the stripping algorithm behavior. Like for the Conv1, bigger tiles capture more zeros, but for Conv2/3, the sparsity gain collapses as the tiles get larger.
3. Precision Comparson: Confirms that moving from FP32 to INT16/INT8 provides a consistent 2x hardware multiplier due to Arria 10 DSP packing.

The most significant finding is the inverse relationship between the Tile size and sparsity gain in the middle layers. In CONV1: a 32x32 tile is the winner. This suggests that the input activations and weigts contain large, contiguous block of zeros (likely due to image background or zero padding). A larger tile allows the stripping algorithm toe eliminate massive cycles in one go.

In Conv2 and Conv3: The 8x8 tile outperforms 16x16 and 32x32. This indicates that these layers are “locally sparse but globally dense.” In a 32x32 block, there is almost always a non-zero value in every row/column, causing the stripping algorithm to fail. The 8x8 tiles are small enough to “find the holes” in the matrix.

The data consistently shows that Total Efficiency doubles when moving from FP32 to INT16/INT8. This is a direct consequence of the Arria 10 Variable Precision DSP block. Because the hardware can pack two 18-bit multiplications into a single DSP, we achieve a baseline 2x speedup before the stripping algorithm even starts. The fact that INT16 and INT8 perform similarly in cycle counts suggests that the current bottleneck is that the number of available DSPs and not the bit-width itself.

We also observe a “U-Shaped” efficiency curve which shows the relationship between Spatial complexity and Network depth. The outer layers (Conv1, Conv5) show a high sparsity gain which handle raw spatial data or final abstract features, both of which are naturally sparse. The middle layers (Conv2, Conv3, Conv4) have low sparsity gain. These layers act as information filters and are much more active (dense), requiring more brute-force computation.

The conclusion for this theoretical data collection is that static hardware design would be highly inefficient. If we were to fix the hardware at 32x32, we would achieve an 86.6x efficiency in Conv1 but drop to a measly 2.3x in Conv2, if we fixed it 8x8, we would get better middle-layer performance but miss out on the massive throughput gains available in the early/late layers.

The NPU controller needs to support Dynamic Tile Resizing in order to maximise the AlexNet inference speedup beyond what any single fixed architecture could provide.

We reach yet another problem however, one that we have seen before with DPR where theory meets the rigid silicon of the FPGA. We cannot change an INT8 into an FP32 DSP at runtime. Instead of one array that changes shape, we could possibly design a multi mode control unit where we synthesise a maximum resolution array. When we move to a layer that only needs 8x8, the controller doesn’t reconfigure the silicon; it simply masks the unused rows/columns and stops feeding them data. The result for this is that you still get the Latency Reduction (your 8x8 tiles finish faster), even if the “Area” (DSPs) is still physically occupied.

So to plan further testing of what we have found so far, we could do some cycle accurate simulation in Questa and then precisely count the cycles between the START signal and the DONE flag. Which could be used to verify the sparsity gain table. And then following that, we could load one single layer’s worth of tiles into the buffer memory (RAM or ROM) and then trigger the systolic array to process that internal data and use the Signal Tap Logic Analyser to capture the output pins of the array and compare them to the python results which would verify the hardware.

Day 32 | 29/01/26 | 6 Hours

Did simulations

Day 33 | 30/01/26 | 6 Hours

Across all the three tests (Conv1, Conv3, Conv5) the latency calculation was accurate, in fact 1 cycle less which could just possibly mean that the control unit just triggered the start of the array one cycle faster than the python model assumed or the output registers are being sampled a cycle earlier.

The 32x32 has ~1.26 speedup, 8x8 had ~1.21 speedup, 16x16 had ~1.24 speedup. These speedups would be a lot more if the test image was less dense but it still made a difference regardless.

Very soon I will need to test the ALM and DSP usage for each of these to try and calculate the costs and possibly a power analysis using power analyser, and then also check out the maximum frequency with the different configurations.

The plan is to switch to the pre-built IP that has the JTAG to Avlon Master Bridge so that I can connect the fpga to my linux laptop which would kind of act like an HPS but without requiring any linux setup that involves the bootloaders. I could just write a simple SystemConsolse (TCL) script to load the matrices onto the fpga. I would simply wrap the top level with a register map ad then connect to my buffers whether thats two or 4 for ping pong and then connect hose to JTAG bridge and control unit. I could test the synthesis results before all this or after, maybe both. So instead of having top level ports M, N, K I would need Mmemory-Mapped Registers just before the input which I have done before in P4P.

A quick note to self here is that the JTAG is a bit-serial protocol which will have lower bandwidth compared to the FPGA processing so it will take much longer to upload the 32x32 matrix from the laptop to the FPGA than it takes for the FPGA to calculate the result. Like the NPU would take lets say 700ns but the JTAG transfer itself might take 50ms. But this is simply just to test the live reconfigurability. – Use the roofline model argument in the report for this I guess.

In terms of what gets changed during this live pipeline for “reconfigurability” the tiling isnt too hard since the M, N K values are just register values that we could change however bit width is something difficult, even with a mux we are still dealing with a default 16-bit physical bus for the Pes if we were to run INT8 we could sign-extend the 8-bit values into 16-bit words and maybe eventually tell the hardware to grab maybe two 8-bit values at once and feed two Pes simultaneously

Day 34 | 01/02/26 | 6 Hours

Day 35 | 02/02/26 | 7 Hours

| Type | Path     | Message                                                                                   |
|------|----------|-------------------------------------------------------------------------------------------|
| ♀ ❌  | 6 Errors |                                                                                           |
| ❌    | act      | Interface with <b>writersresponsevalid</b> requires <b>response</b> signal                |
| ❌    | act      | Has write responses but no <b>write</b> or <b>writedata</b> signals                       |
| ❌    | act      | Read interface must have <b>readdatavalid</b> when <b>writersresponsevalid</b> is present |
| ❌    | weight   | Interface with <b>writersresponsevalid</b> requires <b>response</b> signal                |
| ❌    | weight   | Has write responses but no <b>write</b> or <b>writedata</b> signals                       |
| ❌    | weight   | Read interface must have <b>readdatavalid</b> when <b>writersresponsevalid</b> is present |

```

└─ clock Clock Input
  └─ clk [1] clk
└─ reset Reset Input
  └─ reset_n [1] reset_n
    <<add signal>>
└─ avalon_slave_0 Avalon Memory Mapped Agent
  └─ avs_address [4] address
  └─ avs_write [1] write
  └─ avs_writedata [32] writedata
  └─ avs_read [1] read
  └─ avs_readdata [32] readdata
    <<add signal>>
└─ act Avalon Memory Mapped Host
  └─ avm_act_address [32] address
  └─ avm_act_read [1] read
  └─ avm_act_readdata [32] readdata
  └─ avm_act_waitreq [1] writersresponsevalid_n
    <<add signal>>
└─ weight Avalon Memory Mapped Host
  └─ avm_weight_address [32] address
  └─ avm_weight_read [1] read
  └─ avm_weight_readdata [32] readdata
  └─ avm_weight_waitreq [1] writersresponsevalid_n
    <<add signal>>
<<add interface>>
```

These errors occurred because the platform designer component editor tool automatically made the wait request signals as writeresponsevalid types which is used for high end systems like PCIe so I just had to change the type to simple waitrequest to clear that error.

Day 36 | 03/02/26 | 7 Hours

Understanding some of the tcl commands for communicating over JTAG to computer:

get\_service\_paths master: Identifies all available JTAG-to\_Avalon Master bridges in the system

open\_service master <path>: Claims the master so that I can start sending data through it

master\_write\_32 <path> <address> <data>: Writes a 32-bit word to a specific memory-mapped address

master\_read\_32 <path> <address> <size>: Reads a 32-bit word from an address, <size> is the number of words to read.

close\_service master <path>: Releases the master when I am finished.

Addresses to use for certain components:

Weight Buffer: 0x0000

Data Buffer: 0x1000

npu registers: 0x2000

Day 37 | 04/02/26 | 7 Hours

Since we can't get the HPS right now I could continue with the JTAG idea and have OPENVINO collect data from the python script and send a command bitstream and data packet to the FGPA's memory in one burst

Day 38 | 05/02/26 | 7 Hours

```
set master_path [lindex [get_service_paths master] 0]
```

```
open_service master $master_path
```

```
master_write_32 $master_path 0x2000 0x20
```

```
master_write_32 $master_path 0x2004 0x20
```

```
master_write_32 $master_path 0x2008 0x3
```

```
set k_val [master_read_32 $master_path 0x2008 1]
```

```
puts "NPU K-Register confirms: $k_val"
```



trying to run the master\_write\_32 command causes a freeze and then closes the channel after trying to run it for 60 seconds and then giving up.

When I try to do a jtag\_debug\_sample\_clock it says it cannot invoke java.util.logging.Logger.severe(String)” because “1’ is null

```
# 2. Get the SPECIFIC path for JTAG Debug (not Master)
set jd_path [lindex [get_service_paths jtag_debug] 0]
set jd [open_service jtag_debug $jd_path]
```

Connection is successful now, it was a small misconnection in platform designer with the master reset driving the reset to the memories and NPU. Moving onto further testing to get closer to goal.

Day 39 | 06/02/26 | 7 Hours

Initially, the NPU was stuck in a busy loop where the ready signal was staying at 1. So then I implemented a hardware decaled handshake where the FSM clears its own start bit upon completion. Handshake was successful where the FSM was now navigating its states from IDLE To DONE.

I also moved away from streaming individual registers to a dedicated Output RAM architecture. This allowed the NPU to dump the the entire matrix into a buffer that the Python/tcl script could read at once, which was just something I had to make in platform designer. After I updated to have the output buffer in the platform designer I had to update the base addresses so the component addresses are now:  
NPU System: 0x3000  
data mem: 0x2000  
weight\_mem: 0x1000  
out\_mem: 0x0000

I had some debug issues where I had a type mismatch between the 2D array and 1D which I just had to change the output signal to 64 bits instead of the 16. I also introduced an intermediate signal to explicitly resize the matrix output into the 32-bit vector required for the Avalon-MM interface since the Avalon bus is 32-bit. These changes made the compilatio time go from 8 minutes to 22 minutes.

Currently facing an issue where the output result is all zeros, so my current hypothesis is that the NPU is trying to calculate 0x0 matrix because the M,N,K configuration registers arent properly catching the values from the tcl script, and the reason may be because the buffers worked in byte offesets but the NPU-wrapper was doing it I symbols which make ignore the writes as I did them in byte inputs (maybe)

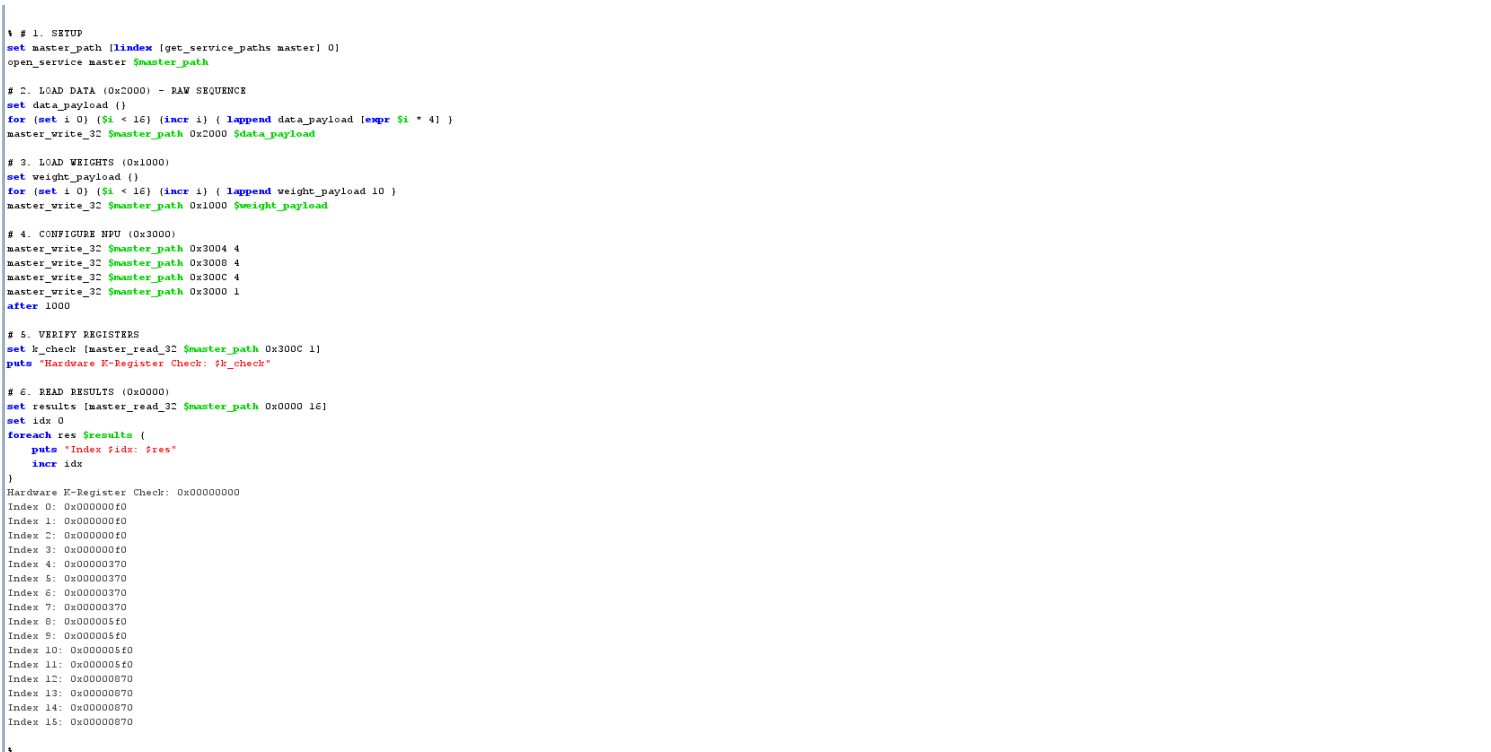
Day 40 | 09/02/26 | 7 Hours

I switched to modelsim based testing and then found some issues with the FSM. The output matrix was all zeros as expected and then also the the first index of the input matrix was undefined and the 2<sup>nd</sup> was 0 and then it would fill up the buffer. To fix this I introduced another middle state that provided a “buffer” cycle to allow the RAM to drive the data onto the bus before the NPU latches it.

After fixing that and some of the waitrequest signals to be initialised to zero, I also needed to fix the timing of some of the flagging signals namely the reg\_ready because that would drive high in the

So once I got all the results done from modelsim I waited for the 22 minute compilation and then tested on the tcl console where I noticed the last element of the 4x4 matrix that I was testing consistently returned 0 on hardware even when the rest of the math was correct. So I found out that the FSM was de-asserting the `avm_out_write` signal at the exact same moment it reached the final count. The avalon slave didn't have enough "hold time" to latch the final word. To solve this I created a `WRITE_FLUSH` state at the end of to hold the final address and data stable on the bus for one extra clock cycle, ensuring the Avalon interconnect completes the transaction before the FSM returns to IDLE.

Day 41 | 10/02/26 | 7 Hours



Day 42 | 11/02/2026 | 7 Hours

Heavily considering switching soon to PCIe where OpenVINO can literally write to a memory address on the CPU side for an insanely high speed transfer because there is no communication because its a direct transfer of data. In theory the PCIe hard IP block inside the Aria 10 receives the the high speed serial bits to rebuild the 32-bit packet that is sent and then converts the request into an avalon\_MM master transaction. Maybe even achieving ~500 nanoseconds per write which would be on par with the NPU processing whereas the JTAG is 1~10 ms per write.

After some thinking and observing the hardware I am provided, I did not have an accessbile PCIe port on the university computer nor my laptop obviously. So then I thought maybe I have to go another route, so I thought maybe Ethernet if I don't need the HPS. If I use an UDP to avalon bridge that would replace the JTAG. However that is an extremely big headache of its own because then I need to get the clock phases right so that data isnt corrupted, the python script would require socket API to send raw UDP packets, so speaking of UDP packets I would need to thinking of a whole data packet system. So I kinda don't wanna deal with tweaking PLL phases, and IP address assignments, packet loss, firewall rules etc. This is because ethernet aint just bits, its layers, because its made for networks.

But for now I guess I can automate the JTAG flow with the pytohn script by loading an image tile

Day 43 12/02/2026 | 8 Hours

We had to find a way to connect the JTAG session with the python script. The idea was to have a tcl script automate the prcoess by capturing the m, n, k values created by the python code for storing in the avalon registers and then also capturing the two matrices in a form of a list.

So I created a python function called `run_jtag_inference` where I was attempting to open up the shell command line from quartus to try and run a pre-coded tcl script and automate the transfer of data from python. However the issue arised from the fact that I wasnt able to connect to the actual console from quartus. When I had the console opened from quartus I noticed it was a Java virtual machine rather than just an embedded executable application like the other quartus services. Which makes things more complicated because I had to find a way to extract a massive library string because the java.exe would be blind to the FPGA hardware without intel's specific .jar files. Even then it could not find the windows .dll drivers for JTAG. So then to fix that I tried injecting bin64 into the PATH and using `.Djava.library.path` which I think did the trick (sort of). But then the isuse was that the command line became too long due to the fact that the .jar files was almost 100 lines of a string and then also the two matrices so to fix that I changed python to write data into a .txt file as a temporary midpoint which tcl would just need to read which I think fixed it because jtag connection was there but it wouldnt calculate anything. So I gave up.

I then thought maybe we could just send through a TCP socket server where the TCL system console could be a listener and python could use socket to send tcl commands as text strings

Day 44 13/02/2026 | 7 Hours

I had given up on trying to fix the unsatisfiedlinkerror in the standalone java.exe because the libraries are probably only designed to link only when initialised by the official quartus environment. Instead I just turned the system console gui itself into essentially a server that watches for files dropped by python. Which isn't the most efficient but it's something for now.

I used tcl commands such as the master\_read\_to\_file and master\_write\_to\_file which would just bypass the java memory limits that I had earlier which would cause crashes. I had to synchronise both the python and tcl to 16-bit signed precision which meant updating the tcl byte count logic to use a \* 2 multiplier for element counts. I also introduced the data packet structure to include a 4 byte aligned header preventing any bit-shifting where an odd-sized header would cause 16-bit values to be sliced across memory boundaries.

I noticed that the zeros I was seeing were most likely caused by the TCL being faster than the disk drive so it would just start the file as soon as it would arrive, which would cause the NPU to just process garbage.

So the current tcl script involves an active while loop within the console that monitors for the .bin file waiting for the full byte count and then performing a hardware handshake by polling register 0x3000 until the NPU clears its "start" bit.

The python function is configured to pack data as np.int16, write it to disk, and then block until result.bin is generated by the FPGA.

Current goal is to debug why those zeros are still occurring and fix it and reduce the mean squared error from -11000 to 0 for bit accurate matching.

Day 45 16/02/2026 | 8 Hours

Spent a lot of time figuring out why the tcl-python automation was failing while the manual tests worked, I added race condition guards to make sure the script doesn't trigger the npu while windows is still halfway through writing the .bin file.

I also noticed that the M N K values are hidden, which my guess was that they were invisible to the read commands.

Day 46 17/02/2026 | 8 Hours

I was able to discover that python was simply just not sending activation files correctly and also the M N K values were being stored in the weight memory registers instead of the NPU's internal registers.

Day 47 18/02/2026 | 9 hours

Morteza meeting points:

- Current pipeline discussion

- Plan for intended pipeline
- 2 Systolic arrays: 16 bit, 8-bit

Day 48 19/02/2025 | 8 Hours

Current plan for the next 3 weeks based on meeting with Morteza:

- Build two systolic arrays side by side where SA1 uses 16-bit DSP blocks, SA2 uses 8-bit LUT-based multipliers. And then use a simple multiplexer controller by a register. So when python sends a "Mode" bit, the hardware simply routes the data to the LUT-based engine instead of the DSP-based one. This allows you to "switch" instantly without a recompile.
- Transform current python code to an OpenVino stack so that it can load the model and extract the tensors for each layer and then pass it all to the current inferencing function that connects to the jtag.
- Also consider using a JTAG UART IP and connect its Avalon slave port to a NIOS V processor's data master and then in C I could use functions to read incoming bytes from python via intel-jtag-uart API
- Consider what tasks to delegate to both python and NIOS V because so far Python would receive the model metadata and perform structured sparsity analysis and then stream those dense tiles to the FPGA via the intel-jtag-uart API. And the NIOS V would wait for those bytes and then write it to the normal buffers and then toggle the NPU start register and then wait for the NPU done flag and then send a finished byte back through the UART to python. The NIOS would be to eliminate race conditions currently seen in the system console. Python could simply just blast the raw stream of bytes into the JTAG UART's internal FIFO, it doesn't wait for a success message for every 32 bits. And if I can move the sparsity analysis that would be even better on NIOS V. But either way NIOS is an end case for the pipeline, everything else needs to be done first.

Currently working on making the OpenVino script. Previously a .pth file was used and the code would catch the data while it was moving. OpenVino works on converting the model into an intermediate representation (IR). In the first stage there is a setup function which takes an xml file that lists every layer and how they are connected. There is also a .bin file which contains the actual weight values in a dense, binary format. It is converted to a FP32 first because it gives us the highest precision possible before I manually cast the numbers to INT16 or INT8.

The next stage involves extraction (function called `extract_layer_data`) where instead of waiting for a forward pass to catch the data, I can tell the OpenVINO engine exactly what I want. I can just pull out a specific layer and its associated weights easily. This is where my research comes handy, we can find the maximum value in the layer, create a scaling factor, and then use `.astype(np.int16)` or `int8`, ensuring that even though the model started as a float, it enters the NPU as a perfectly scaled integer and is perfectly reconfigured to process that particular tile at hand.

This next part is just my old python jtag inferencing function which handles the tiling and alignment. Since the matrices aren't usually a simple 32x32 I can just take that tile and then stamp it onto a zeroed-out 32x32 Numpy array. And by padding it to exactly 32, I can ensure that when the VHDL code jumps to row 1, it lands exactly on the 32<sup>nd</sup> word in memory, because if I didn't do this, the npu would read row 1 weights as row 0 inputs, causing MAC discrepancies.

Day 49 | 20/02/2026 | 8 Hours

Honestly since I kept having wrong outputs from my hardware but it was fine for smaller matrices I feel like I am reaching the limit to what my JTAG-To-Avalon bridge can do. The drift after row 6 and now at this point row 0 I believe the errors are caused by either mismatch in sparsity handling or JTAG latency. And since I have quite a lot of verification done in sparsity handling I would think it is JTAG latency. So this would mean that the NPU is running faster than the JTAG can feed it, causing the master to grab whatever happens to be in the FIFO, leading to the “skipped row” effect.

So I think it is time I stop wasting my efforts on further trying to verify the NPU hardware because it does work for sure so I need to move onto producing my outputs from the openvino script I made and then also work on creating NIOS interface. Hopefully NIOS will be better because with TCL it would just wait and not know if the NPU had actually finished or if the JTAG bridge stalled.

Day 50 | 22/02/2026 | 2 hours

Had to verify the openvino based python script that I made a few days ago:

- The im2col transformation for the first convolutional layer resulted in a matrix of  $M = 3025$ ,  $K = 363$ , and  $N=64$ . This confirmed that the mapping from the cat.jpg image into the flattened buffer format was dimensionally correct for the systolic array.
- The biggest technical hurdle was the numerical overflows and data types. To elaborate, the output matrix for the top 4x4 tile showed massive negative numbers in the hundreds of millions. This was due to the fact that I was using 16-bit signed integers for multiplications. So when performing the dot products for the AlexNet kernels, the 16-bit values were overflowing the 32-bit accumulation boundary, leading to the corrupted results. We established that we need to strictly manage the bit-growth during the convolution or use a scaling factor before the data hits the NPU.

Day 51 | 23/02/2026 | 9 Hours

I spent the day working on creating a NIOS processor with a JTAG uart component. There were a few hiccups with the addressing for all the components in the platform designer so I had to spend a lot of time playing around with that. And then once everything was made and recompiled, it wouldn't program the board due to some CONF\_DONE pin being low or something. Turns out I just needed to reload the .sof file.

After all that I went to try and run a basic project in Ashling Riscfree IDE but it was a lot more complicated than NIOS II eclipse days because intel decided to change the way that the IDE works. So I had to get all the BSP and make files stuff working in the nios V shell cli. After all that I ran the hardware debugger in the IDE but it would just print out some debugger information and not hello world. I then found out that I have to run the jtag uart terminal in the shell because the system console inside the Ashling IDE does not show JTAG output which is stupid but it's what intel has done.

Day 52 | 24/02/2026 | 8 Hours

I did some other testing to see the communication and I could send some bytes over and then nios would send back so that confirmed that the intel-jtag-uart python module was working.

I extended that NIOS C code to perform direct memory-mapped writes to the NPU system using volatile pointer access, where I configured the M, N, K values then the memory registers followed by the ready bit. I conducted a 4x4 matrix multiplication test directly from nios to the NPU hardware. For this, the results were verified to be 100% bit-accurate against software matmul reference.

Day 53 | 25/02/2026 | 9.5 Hours

Even a basic 32x32 was successful with the npu when done just from nios and checked in tcl script. However, when python connection was added into the mix, it became extremely problematic. I tried various byte ordering fixes, FIFO flow control via polling and chunked transfers with acknowledgement handshakes. All failed because JTAG UART has a 64 byte FIFO which is essentially a debug interface and not designed for bulk data. So I reverted back to TCL/JTAG Master for data transfer. Memory addresses had been shifted around after NIOS addition.

Moving onto pure TCL scripting, the FSM was reading at 32-element row boundaries expecting NxN padded layout, causing row misalignment from row 1 onwards and zeros beyond row 16. An attempt to fix the FSM by changing the counter bound to  $m*k$  broke the weight fetching since both data and weight shared the same counter. So then I just kept it as is and did 32x32 padding which was confirmed in simulation.

Questions for morteza:

- How extensive should the research report lit review be? Given that most of my research was done in p4p.

Day 54 | 26/02/2026 | 11.5 Hours

The fixes in the previous days and its outputs were simply false positives and something deeper was at play.

I noticed that the problem was that data matrix is  $M \times K$  shaped and weight matrix  $K \times N$  shaped. So both have different dimensions, nothing new about this. But since the same counter and indices were being used for both, the indexing worked correctly for the data matrix but would drift out of sync for the weight matrix after a certain depth. Specifically rows 6-9 were corrupted – the first few rows matched but then values went wrong.

The root cause was that when `col_idx` wrapped at `reg_k` for the data matrix, that same wrap was being applied to the weight matrix which should have wrapped at `reg_n`. So weight matrix columns were indexed incorrectly from row 1 onwards, loading wrong values into `n_matrix_weight`.

The fix was to split into two sequential fetch phases. First fetch data  $M \times K$  with `col_idx` wrapping at `reg_k`, then reset counters and fetch weight  $K \times N$  with `col_idx` wrapping at `reg_n`. Each phase uses independent counters and addressing so the shapes are handled correctly. That's what introduced the `FETCH_DATA` and `FETCH_WEIGHT` state split.

Day 55 | 27/02/2026 | 11 Hours

Day 56 | 02/03/2026 | 7 Hours

I had the fortunate moment of realisation the day prior which was a Sunday my day off that my NPU was addressing in words and not symbols (bytes). This would make sense why it seemed like the memories seemed to be aliasing and it actually has occurred before but I never cared to check my logbook if I had this issue before because I had completely forgotten. Good news was that I got to keep the changes in my `npu_wrapper_system` script the same where I hardcoded the memory addresses. I also made sure to compile with a signal tap file alongside it so just in case something went wrong I would have an easier time debugging.

I recompiled and the matrices were finally computing again but it seemed wrong past the 1<sup>st</sup> row. The problem I created was that my FSM was now fetching  $M \times K$  words for data and  $K \times N$  words for weights but memory had them stored as 32-wide rows. So this meant that the FSM was reading them at the wrong stride. So I simply just made it that tcl and python will no longer do any padding and just send compact tiles which I only did because I was lazy for recompiling, but at the same time it was kind of aligned with how my control unit works since it took  $M \times N \times K$  values and the FSM would just load the tiles into the particular index sequentially, it doesn't care how it arrived in memory, so in theory sending the tight-packed TCL read should make sense.

And what do you know, boom I had 100% bit-accuracy and had a proper pipeline from quantised Alexnet → Sparsity extraction → Compact Tile → JTAG → NPU → Python Verification.

So now I could finally move onto my main plans where I have my two systolic arrays. I just have to make some changes to the current system and make some new files. I would just have to make array size a generic  $N$  and duplicate for INT8 version with LUT inference attribute. And if possible change the wrapper to handle packing two 16-bit and 4 8-bit which would simply be controlled by a mux on output and a config register. I will also need to update the Python/Openvino to decide per layer which config to use, pack data correctly, send config byte in header

- Just an update on the resource usage right now pre changes I notice that it has only use 519/1687 DSP blocks and 84k/251k logic utilisation in ALMs.

As per my changes I had some doubt that there may be some mess ups between layer transfers if not synchronised correctly. But after some thorough checking, I see that each tile would be independent and when the Convolution layer in question will finish, the FSM should just go back to IDLE and the ready flag should clear, all Pes reset. So if a new convolution does start, its a completely fresh run with new data new weights, new config. The only state that persists between tiles is whatever is left in the PE accumulators – but those get reset via `n_reset` which fires on `reset_n` deassertion, and the PE enable signal only allows accumulation when the correct tile is running. So all I would need to do is be careful that on each tile that is running, the `reg_config` is configured correctly

A testbench I made was successful in calculating both the 16-bit and 8-bit back to back. All I needed was having 8-bit versions of the hardware and some updates in the actual NPU hardware. The `NPU_system_wrapper` now has a new `reg_config` register where 0 represents INT16 and 1 represents INT8. Added an `n_done_mux` which selects between `n_done` and `n_done_int8` based on `reg_config` and all the other fetching based states now branch on `reg_config`.

So far I did not follow through for my plan for having the two 16-bit values per 32-bit word and 4 8-bit values per 32-bit word which would increase my throughput quite a bit. However, some changes would be required. The FSM fetch counter would increment differently since it would have to unpack either either 2 or 4 values per fetch. So it gets tricky when the row ends mid-word. So if INT16 with  $K=5$  the



2<sup>nd</sup> word contains elements 4 and 5, but elements 5 belongs to the next row. I would need to handle case where a packed word straddles a row boundary. If K is odd for INT16 or not divisible by 4 for Int8, the last word of each row has partial valid data and partial garbage from the next row or padding.

Python packing must match exactly: If python packs row-major tight with no padding between rows, the FSM must know when word crosses a row boundary. If python pads each row to an even number of elements (for Int16) or multiple of 4 (for Int8), the FSM is simpler but wastes memory. So the simplest and safe approach is to pad the memory.

Day 57 | 03/03/2026 | 8 Hours

Effectively tested for Conv1,3,5 and it works perfectly. However I noticed that since the last layer worked best with 16x16 INT16 I was still using the 32x32 hardware where 16x16 Pes are enabled. So for now I will keep as is.

I need to plan out the metrics I need to calculate by the end for report data.

1. DSP blocks
2. ALM/LUT count
3. Memory blocks
4. Maybe FMAX but that should just be an aim to be as high as possible
5. Cycles per tile
6. Throughput in MACs per second
7. Quartus Power analyser – Compare static and dynamic power between single array and dual array
8. This is important because the Int8 LUT array should consume less dynamic power than DSPs for small tiles.
9. Bit accuracy percentage per layer but that's probably just initial testing because I know it works perfect now there shouldn't be any edge cases
10. MACs per DSP block
11. MACs per ALM

I decided to view the timing analyser so that I can potentially increase the clk frequency. Currently there are no failing paths that show up so it's hard to see if anything needs to be fixed/adjusted. However seeing that the fmax on the worst case is 69.66Mhz on a 50Mhz clk it can never achieve 100Mhz without making some serious changes. This is because the timing summary shows a slack of 14ns for a cycle of 20ns. So a 100Mhz clk would have a cycle of 10ns and obviously 14ns exceeds that I would end up having -4 slack. After observing the logic depth which refers how many look-up tables a signal must pass through between two registers. So after the clk source it shows 2 signals going through 2 levels, 52 signals going through 5 and 946 going through 6 levels. The bottleneck then becomes evident, I have nearly thousand signals that must traverse 6 levels of logic in a single clock cycle and

routing delay typically takes about 1.5-2ns so 6 levels would mean 12ns per cycle of just routing delays.

Day 58 | 04/03/2026 | 9.5 Hours

Not much could be told in the timing analyser yesterday but then today I looked at the rpt files and noticed that the source register for `w_row_idx[4]` to the destination register `avm_out_writedata[22]` was most likely causing the 6 logic level routing, they are pretty much synthesised intermediate LUT nodes quartus created to implement the array indexing logic. The massive IC delays jumping between physically distant locations in the chip told the story pretty much. `w_row_idx` is a register sitting at one corner of the chip, and `n_output` results are spread across the entire FPGA because there are 1024 PEs placed all over it. The mux that selects the right PE result based on `w_row_idx` and `w_col_idx` has to route signals across huge physical distances hence 13.282 and 14.457 ns delay being pure interconnect.

So then I made it so that the original `WRITE_RESULTS` state is split into two. `WRITE_RESULTS` now only reads from `n_output` and registers the value into `out_data_Reg` so that no Avalon Write happens here. `WRITE_AVALON` which is the new one, takes the pre-registered `out_data_reg` and actually writes it to the Avalon Bus. The reason is that the critical path was `w_row_idx` → `n_output` array mux → `avm_out_writedata` all in one cycle, so it'll be two shorter paths instead of one long one.

I made sure to do some simulation testing beforehand. The reports for the result in the transcript seem to be wrong but this is why I should also check the waveform as well because that did show the correct outputs and states so it was a false negative, so lesson is to always triple check before I attempt to "fix" things.

I also saw that the potential reason that my register reads for status and everything else was not readable since it gets registered to the value from the previous cycle which was literally just cleared to zero in the beginning of the current. So the fix is to move `avs_readdata` out of the clocked process and make it combinational.

Whilst that compiles I decided to review the OpenVino script to see what needs to be done as I will be doing a final switch from the standard pytorch. So far the OpenVino script has not been touched in a while and so it seems very outdated:

1. It uses manual quantisation scaling, but I should be using the given quantisation API
2. So far it only handles Conv1 hardcoded so I need to make it iterate all conv layers.
3. No config decision logic
4. no binary file pipeline
5. no sparsity handling and data collection

I had a look at some API:

1. `ov_model.get_ops()` - iterate all ops to find layers by type
2. `op.get_type_name()`
3. `op.input(1).get_source_output().get_node().get_data()` - extract weights from conv op inputs directly.
4. `core.compile_model()` with `ov.properties.hint.inference_precision` – run inference and extract intermediate activations via output tensors.

Upon “research” discoveries, I see that not each vision model will have the same naming convention for convolutional layers so the most robust approach is to write a model-agnostic layer discovery function that would

1. Load any model via `ov.convert_model` from pytorch
2. Scans all ops for Convolution type
3. Marks each conv output as a model output
4. Runs inference once to get all activations simultaneously
5. Extract weights directly from the conv op’s weight input.

And another big reason for moving to OpenVINO is the ease of metadata collection that I could extract for my research and NPU processing:

1. Per-layer metadata from OpenVINO graph:
  - Kernel size, stride, padding, dilation – extracted directly from the convolution op attributes
  - Weight shape (out\_channels, in\_channels, kH, kW) – from weight node output shape
  - Input feature map partial shape – from op input 0
2. Per-tile dynamic decisions:
  - Activation sparsity and weight sparsity at layer level
  - Tile-level sparsity after `coordinated_row_removal`
  - Config decision (int8 vs Int16) based on actual tile dimensions and value range
  - Every tile independently decides – same layer can mix Int8 and Int16 tiles depending on sparsity.

So the final summary would give me:

Total tiles, match rate, Int8 vs Int16 split, average sparsity – all the metrics I need for my report.

Write about all stuff I did to solve the overflow issue because I have to truncate my 64bit accumulator to 32bit bus

After viewing the compilation reports for the new compile I see that the fmax is now 83Mhz but thats barely an increase that I wanted but I have discovered some new data that can help me fix this. There is another critical path `w_col_idx` that has a 12nd data delay which is causing issues with the `WRITE_RESULTS` state. I will have a look at that later because there are some other stuff that needs to be addressed first. I can see all the nodes of each logic depth:

So sitting at depth 8 is the `max_run_cycles` from the control unit which is a combinational signal that feeds idrectly into the `count >= max_run_cycles` comparison inside the control unit process. So every cycle the comparator recomputes this value combinationally and comapres it against count thats 8 levels of logic because its an adder feeding a comparator, so it needs to be registered.

Sitting at depth 6 is the source of the address multiplier which computes `write_counter * 4`. This is kinda weird because the multiplier output is feeding back into the row index somehow. This suggests the `write_counter * 4` multiplication in `WRITE_AVALON` is creating a long path because integer multiplication of a counter is being implemented in LUT logic rather than using a DSP or maybe

because its chosen the worst case path which uses LUTs because of 8-bit SA im not sure. I might try a left shift for the write\_counter so that it doesn't do any hardware stuff, its just wire routing.

Sitting at depth 5 is just the row col index stuff we need to change so thats already discussed.

Sitting at depth 4 is the same as depth 6 just shorter paths so I guess that confirms that the write\_counter \* 4 multiplication is a recurring problem across multiple path lengths.

Depth 3 is chillin tho its just data loading paths and the timing itself is pretty close to 10ns so its almost meeting 100Mhz anyway without any changes.

RTL fixes:

- Control unit max\_run\_cycles was changed from concurrent assignment to registered inside clocked process when ready = '1', breaking the depth 8 combinational path.
- Wrapper w\_row\_idx\_reg/w\_col\_idx\_reg: added register copies updated one cycle ahead in WRITE\_AVALON, WRITE\_RESULTS uses \_reg versions for array indexing, breaking the depth 5 routing path from counter increment to 32x32 output array mux
- PE elsif en = '0' accumulator clear – clears accumulator, multiplier, data and weight registers when enable goes low.

Did not compile tho because I needed to test out the thing.

Eventually when I could test it, I saw that it gave an fmax of 87hz, depth got reduced from 8 to 5. The control unit fix worked. The PE fix appeared to work in RTL but didn't fix the hardware problem because en was never actually going low, that root cause wasn't discovered until after the compile.

OpenVINO pipeline work after bit-accurate single tile match:

First attempt at full inference: I switched from the run\_single\_tile to main with t\_size=32. Immediately hit KeyboardInterrupt at time.sleep(0.1) in the result.bin wait loop. This was because tile.bin was not being deleted by TCL after processing so python kept writing new tile.bin but TCL kept reprocessing the same one or ignoring. Fixed by confirming file delete \$bin\_path was in TCL.

Status polling TCL: Replaced the fixed after 2000 wait with a proper done polling loop reading 0x30014 every 10ms, breaking when bit 0 goes high, 500ms timeout as safety. Done detected after 0ms confirming the NPU completes very fast and polling works correctly.

Full inference attempt: Ran main for AlexNet. Tile 1 matched perfectly. Tile 2 through 48 all mismatched with exactly 1024 non-zero diffs for full 32x32 tiles and proportionally fewer for partial tiles. This consistent pattern across every single tile after the first immediately suggested systematic accumulation rather than random error. I added a "diff sample" printing to the mismatch block. Discovered that tile 3 diff sampel was exactly the negative of tile 2 hardware result, tile 4 diff was exactly the negative of tile 3 hardware result, and so on. This was the textbook evidence of the accumulator not resetting, each tile's hardware result is the correct answer plus all previous tiles' reuslts accumulated on top.

So at an attempt to fix this I initially suspected the PE accumulator wasnt clearing because reset was never asserted between tiles, only reg\_ready pulsing. I added an elsif en = '0' statement to clear the accumulator and then realised the testbench ran INT16 then INT8 back to back successfully but then also realised that that proved nothing because they were 2 different SA instantiations with two different

accumulators. Wrote a TCL script running the same 4x4 identity matrix test twice consecutively for Int16. Results showed row 0 correct btoh times initially, rows 1-3 doubled. Then when I ran it again it kept accumulalting. So then I looked at the systolic array code, 'en' for each PE is driven by `enabled_PE(i,j)` which comes from `mask_internal` in the control unit. `mask_internal` is only set when `ready = '1'` and `count = '0'`. It is never cleared after the tile completes. So `en` stays permanently high for all active PEs after the first tile. The PE `elsif en='0'` clause never triggers because `en` never goes low. The accumulator runs continuously.

I Thought of fixing the control unit where `run_enable` goes low at tile completion, simultaenously clear all `mask_internal` entries to zero. This drops `en` on all PEs the cycle the tile finishes, the accumulator clears on the next clock edge, and the next tile starts clean.

This compile contains: Control Unit mask clear, control unit int8 mask clear, PE accumulator clear caluse, `max_run_cycles` registration, wrapper `w_row_idx_reg` fix. All five changes in one compile.

Still at depth 4 paths around 9.5ns from DSP output to `w_row_idx` duplicates. These are routing distance problems not logic depth but will fail at 100Mhz if I tried so. Which I will address after fixing the accumulator

Day 59 | 5/03/2026 | 9 Hours

After a bunch of back and forths I just connected a signal from the wrapper to the NPU cores as a proper reset. So this would just reset everything with full determinacy.

After adding the reset the `fmax` dropped from 87 to 79Mhz because the reset logic in wrapper adds a logic level to the reset path so every register in both NPU cores now has an extra OR gate on its reset output. That fans out to thousands of registers and adds routing pressure everywhere.

I did get to succeed on full inference but it is extremely slow which I cant do much about cause of JTAG.

To collect data around this bottleneck effectively I will most likely just measure the cycle count per tile from `n_cycle_count` which is already in the hardware. For each tile I know that `M`, `N`, `K` so that I know that exactly how much MACs were performed:  $2 * M * N * K$ . Divide by cycle count to get MACs per cycle. Multiply by `Fmax` to get GOPS. That's your actual hardware throughput measurement – no JTAG timing involved, just cycle counts from the hardware itself.

Then I would compare the two scenarios: 1 without sparsity exploitation with full tiles and 1 with exploitation with actual `M N K` after zero/column removal.

The ratio of MACs saved is your efficiency gain. That's a concrete, measureable speedup that comes directly from the hardware runs.

For latency I would extrapolate with `X` MHz `fmax` with `Y` cycles per tile and `Z` tiles per layer, theoretical layer latency would then be  $(Y * Z) / X$ . And then do this per layer and sum for full model

latency. That would be my projected inference time on a proper high-speed interface, not through JTAG.

### Something about a roofline model

Day 60 | 06/03/2026 | 7 Hours

After trying a full AlexNet inference I see that it always has full tiles with very minor sparsity percentages for activations and weights. After careful evaluation I notice that it is because of how openVINO extracts activations. The activations I am getting are pre-ReLU in the deeper layers – Alexnet applies ReLU after conv, but openVINO is giving the conv output before activation, so that means I'm seeing the full dense output with positive and negative values, not the sparse post-ReLU activations which are where the zeros come from.

For weights, AlexNet's pretrained weights are dense by nature – it wasn't trained with any sparsity regularisation so there's no reason for weights to be zero. This would make sense if I'm using raw OpenVINO IR activations for sparsity analysis, my previous PyTorch script was likely capturing post-ReLU activations which is why I saw more sparsity.

For activations I will just apply ReLU after each conv activations so that I can get proper data for inference but as for the weights, I could technically find a pretrained pruned version of Alexnet and others weights or I could manually zero out some filters but that would be data manipulation right there so no.

Day 61 | 09/03/2026 | 7 Hours

Added the 3<sup>rd</sup> instantiation for SA: 16x16 INT16

Day 62 | 10/03/2026 |

Report Writing

Day 63 | 11/03/2026